The State Law for Lazy-Point Train Tracks: $f(N) = \min(2^N,\ N + 4)$

Carl Heise*

6 September 2026

Abstract

A single train travels over a layout of N three-way switches with *lazy points*: entering at the stem, the train follows the current tongue and moves nothing; entering at a branch, it forces the tongue to that branch and leaves by the stem. How many distinct settings of the N tongues can one train ever exhibit? Exactly

$$f(N) \ = \ \min\bigl(2^N,\ N + 4\bigr) \qquad \text{for every } N \geq 0.$$

The bound is uniform over all layouts, all starting positions, and all initial tongue settings; it is attained, for every N, by an explicit layout. The whole law is machine-checked as a single Lean 4 theorem, `GeneralN.state_law`, built without Mathlib, whose axiom audit is exactly the three standard Lean axioms. This document is a human-readable rendering of that proof: complete for the model and every attainment construction, and a structured account of the sharp $N + 4$ upper bound.

Contents

*Human-readable rendering extracted from the machine-checked Lean 4 development (`github.com/senegrom/duplotrain`, directory `theory/lean`); prepared with the assistance of Claude (Anthropic). The formal proof is the authority: every numbered statement below is a theorem of that development, and the sharp upper bound (Section 5) is presented as a faithful account of the formal argument rather than an independent proof.

1 Introduction

The question comes from a toy: LEGO® DUPLO® train track. Its Y-switches have unsprung, *lazy* points. A train entering the switch at its single stem is routed by whatever position the tongue happens to be in, and the tongue does not move. A train entering through one of the two branches pushes the points aside if necessary — the tongue ends up aimed at the branch the train came in by — and exits through the stem. A layout wires the free ends of switches to one another with plain track; plain track has no state, so the machine state of a layout is the vector of its N tongue positions, one bit per switch.

A single train, placed anywhere, either eventually falls off an unwired end or drives forever. Along the way the tongue vector changes. The *state count* of a run is the number of distinct tongue vectors it exhibits; $f(N)$ is the maximum state count over all N-switch layouts, all starts, and all initial tongue vectors. Trivially $f(N) \le 2^N$. The surprise is how far below 2^N the truth lies:

Theorem 1.1 (The state law)**.** *For every $N \ge 0$,*

$$f(N) \;=\; \min\bigl(2^N,\; N + 4\bigr).$$

That is: no run on any N-switch layout ever exhibits more than $\min(2^N, N+4)$ distinct tongue vectors, and for every N some layout, start, and initial setting exhibits exactly that many.

One train can barely move the machine it drives over: of the 2^N conceivable switch settings, all but $N + 4$ are unreachable within any single run, no matter how the track is laid. The 2^N leg of the minimum binds only for $N \le 2$ ($f(0) = 1$, $f(1) = 2$, $f(2) = 4$); from three switches on, $f(N) = N + 4$.

The result was found and proved formally. A sequence of machine-checked bounds

$$26N{+}3 \to 24N{+}5 \to 18N{+}3 \to \cdots \to 2N{+}9 \to N{+}7 \to N{+}6 \to N{+}5 \to N{+}4$$

was tightened until the constructive lower bound was met. The final development (6,649 lines of Lean 4 in 31 files, using no external mathematics library) proves the single statement

$$\texttt{GeneralN.state\_law : } \forall N,\ \texttt{IsExactStateCount}\ N\ (\min(2^N, N+4))$$

and audits it to exactly the axioms {`propext`, `Classical.choice`, `Quot.sound`} — in particular with no appeal to native code evaluation: the few finite computations are checked by the proof kernel itself.

What this document is. Sections 2–4 are complete human mathematics: the model and all attainment constructions, each of which a reader can verify by hand. Section 5 renders the sharp upper bound $f(N) \leq N + 4$ — by far the deep half, the majority of the formal development — as a structured account: the objects, the main lemmas, and the charging argument, with enough proof for each step that the shape of the mathematics is visible, and with pointers into the formal text, which remains the complete reference. Section 8 records what was formalised and how to check it.

The physical-trace core of the argument (Section 5.2) descends from the first-repeated-track idea in the train-set literature; the formal sources name Chalcraft–Greene and Aaronson.

2 The model

Everything is stated over a discrete dynamics in which plain-track geometry provably never enters; only the wiring pattern matters.

Definition 2.1 (Ports and wirings)**.** Switch k ($k = 0, 1, 2, \dots$) owns three *ports*, encoded as natural numbers: its *stem* $3k$, its *left branch* $3k + 1$, and its *right branch* $3k + 2$. A *wiring* is a partial map w from ports to ports that is symmetric ($w(p) = q$ implies $w(q) = p$), recording which port is joined to which by plain track; a port with no partner is an unwired end. A layout *uses only switches* $0, \dots, N - 1$ when every wired port is smaller than $3N$.

Nothing forbids joining two ports of the same switch (the lobes of Section 3), nor $w(p) = p$: a port joined to itself makes the train re-enter the port it has just left, which is exactly a dead end that reverses the train (in the toy, a buffer with a direction-change stone). The upper bound therefore covers reversing terminators, while every attainment layout below uses ordinary two-ended track only.

Definition 2.2 (Tongues, states, the step)**.** A *tongue vector* is a function u from switches to $\{\mathsf{L}, \mathsf{R}\}$. A *state* is a pair (p, u): the train is about to enter port p with tongues u. One *step* first performs the local passage,

$$\mathrm{arrive}(u, p) = \begin{cases} \big(\text{branch of } k \text{ selected by } u(k),\ u\big) & p = 3k \text{ (facing entry)}, \\ \big(3k,\ u[k \mapsto \text{the branch } p]\big) & p \in \{3k{+}1, 3k{+}2\} \text{ (trailing entry)}, \end{cases}$$

producing an exit port and possibly one changed tongue; the train then travels over the exit port's plain-track edge, so the next state is $(w(\mathrm{exit}), \cdot)$ if the exit port is wired and the run *dies* otherwise. We write $\mathrm{step}^k(c)$ for k iterated steps from state c (undefined once the run dies), and call time k *live* if $\mathrm{step}^k(c)$ is defined.

Definition 2.3 (Restricted vectors and the exact count)**.** For a run from c_0 on a layout using switches $0, \dots, N - 1$, the *restricted tongue vector at time* k is the list $\big(u_k(0), \dots, u_k(N{-}1)\big)$ of the first N tongues at time k. For $N, c \in \mathbb{N}$, say c is the *exact state count* for N (`IsExactStateCount` N c) if

- (*upper bound*) for every wiring on switches $0, \dots, N-1$, every start, and every duplicate-free list of live sample times whose restricted vectors are pairwise distinct, the list has length at most c; and

- (*attainment*) some wiring, start, and such a list attain length exactly c.

Theorem 1.1 is then literally $\forall N$, `IsExactStateCount` N $(\min(2^N, N+4))$, with no side condition: $N = 0$ is witnessed by the empty layout.

Remark 2.4. Sampling distinct vectors at chosen live times is the right phrasing of "the run exhibits c distinct vectors": it makes the upper bound a statement about every finite observation of the run and the lower bound an explicit finite certificate.

3 Attainment for $N \le 2$

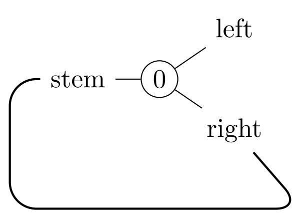
Here $\min(2^N, N+4) = 2^N$: *every* tongue vector must be visited. Each witness below is a finite run; in the formal text these are checked by the proof kernel's `decide`, and the reader can replay them from the tables.

3.1 $N = 0$: the empty layout

With no switches there is one restricted vector, the empty one, and the layout with no track at all exhibits it at time 0. So $f(0) = 1 = 2^0$.

3.2 $N = 1$: the teardrop

Construction 3.1. Wire the stem of switch 0 to its own right branch ($w(0) = 2$), and leave the left branch unwired.



stem–right teardrop; left branch capped

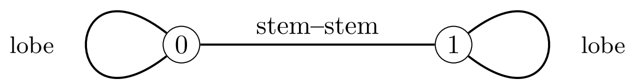
Start the train entering the right branch (port 2) with tongue L. The trailing entry pins the tongue to R and exits the stem; the stem's edge leads back into port 2. The two sampled times already show both vectors:

time	entering port	tongue of switch 0
0	2	L
1	2	R

So $f(1) = 2 = 2^1$. (Thereafter the vector stays R: the run is live forever but mints nothing new — a first taste of the upper bound.)

3.3 $N = 2$: the dogbone Gray square

Construction 3.2. Join the two branches of switch 0 to each other ($w(1) = 2$), the two branches of switch 1 to each other ($w(4) = 5$), and the two stems to each other ($w(0) = 3$).



Each branch-to-branch loop is a *lobe*: a train leaving the stem comes around the lobe and re-enters through the *other* branch, trailing, which flips that switch's tongue — lobes alternate. Start the train entering the stem of switch 0 with both tongues L (write a vector as the pair $u(0)u(1)$). Every second step completes one lobe pass and flips one tongue, walking the full Gray cycle of the 2-cube:

time	entering port	vector
0	0 (stem 0)	LL
2	3 (stem 1)	RL
4	0 (stem 0)	RR
6	3 (stem 1)	LR

(The odd times enter the lobes' far branches: ports $2, 5, 1, 4$.) All four vectors of two switches appear, so $f(2) = 4 = 2^2$. The next flip returns to LL at time 8: the run cycles through the square forever.

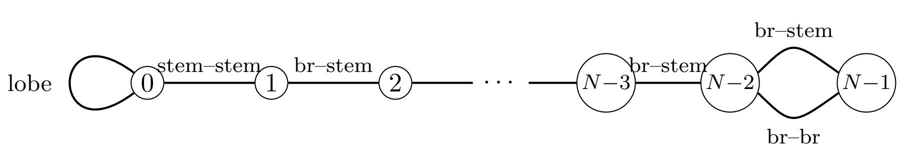
4 Attainment for $N \geq 3$: the extremal family

Now $\min(2^N, N + 4) = N + 4$, and one explicit family of layouts attains it for every $N \geq 3$ (`state_law_lower_bound`, file StateLawLowerBound.lean; found empirically by a Python state-count prober, then proved symbolically in N).

Construction 4.1 (The extremal wiring)**.** On switches $0, \ldots, N - 1$:

- switch 0 carries a lobe (its two branches joined, $w(1) = 2$), and its stem is joined to the stem of switch 1 ($w(0) = 3$) — together a teardrop hanging at the bottom;
- switches $1, \ldots, N - 3$ form a chain: the right branch of switch k is joined to the stem of switch $k + 1$;
- switches $N - 2$ and $N - 1$ are doubly linked: the left branch of $N - 2$ to the stem of $N - 1$, and right branch to right branch.

All other ports are unwired.



Theorem 4.2 (Lower bound, $N \geq 3$)**.** *On the wiring of Construction 4.1, the cold start — entering the right branch of switch $N-2$ with all tongues* L *— is live at the $N+4$ times*

$$0,\ 1,\ \ldots,\ N-2,\quad N,\quad 2N-1,\quad 2N,\quad 3N-1,\quad 4N-1,$$

and the restricted tongue vectors at those times are pairwise distinct. Hence $f(N) \geq N+4$.

Proof (rendered). Identify a vector with its set of R switches. The run has four phases, and the sampled vectors are:

time	vector (R-set)
$0, 1, \ldots, N-2$	$\varnothing,\ \{N{-}2\},\ \{N{-}3, N{-}2\},\ \ldots,\ \{1, \ldots, N{-}2\}$
N	$\{0, 1, \ldots, N{-}2\}$
$2N-1$	$\{0, \ldots, N{-}1\}$
$2N$	$\{0, \ldots, N{-}1\} \setminus \{N{-}2\}$
$3N-1$	$\{0, \ldots, N{-}1\} \setminus \{N{-}2, 0\}$
$4N-1$	$\{0, \ldots, N{-}1\} \setminus \{0\}$

Phase 1 (descent, times $0, \ldots, N-2$). The cold train enters the right branch of $N-2$ trailing, pinning $N-2$ to R and leaving by its stem, which is wired to the right branch of $N-3$; the trailing cascade rolls down the chain, pinning one new switch per step — each sampled prefix mints a new vector.

Phase 2 (teardrop, times $N-1, \ldots$). From the stem of switch 1 the train reaches the stem of switch 0 facing. Its tongue says L, so the train exits by the left branch into the lobe and comes back through the right branch, trailing: switch 0 flips to R (the time-N vector). Emerging from stem 0 it re-enters stem 1 facing; each chain switch's tongue now points at the branch the train arrived by in Phase 1, so the train retraces the chain *backwards by pure facing moves, changing nothing* (the retrace mechanism of Section 5.1), reaching the stem of switch $N-2$ at time $2N-3$. Its tongue, set to R in Phase 1, sends the train out the right branch, and the right–right link delivers it into the right branch of switch $N-1$ at time $2N-2$.

Phase 3 (closing the far switch). That trailing entry sets $N-1$ to R: at time $2N-1$ all N switches are R. The train leaves by the stem of $N-1$, whose link leads into the *left* branch of $N-2$, trailing — so $N-2$ flips to L at time $2N$ (the train now heading down the chain again).

Phase 4 (the Gray oscillation). From here the run oscillates on the two coordinates $N-2$ and 0 only: down the chain (trailing entries that change nothing), around the teardrop lobe — which flips switch 0 — back up by facing retrace, and around the double link — which flips $N-2$: the tongue of $N-2$ decides whether the train arrives at $N-1$ through its stem or its right branch, and either way it returns into the other branch of $N-2$. Concretely, 0 flips to L at time $3N-1$, $N-2$ back to R at $4N-1$, 0 back to R at $5N-2$, and so on with period $3N-1$: the four corners of the square are visited in Gray order, and times $3N-1$ and $4N-1$ sample the two corners not yet seen.

Distinctness is a matter of pointing at one differing coordinate for each pair, and the table makes the witness explicit: within Phase 1, coordinate $N-1-m$ separates the m-th prefix from the later ones; coordinate 0 separates Phase 1 from time N; coordinate $N-1$ separates times $\leq N$ from times $\geq 2N-1$; and the four late vectors differ on $\{N-2, 0\}$, where they realise the four combinations. (In the formal text the trajectory claims are proved by induction on each phase and the liveness and distinctness checks are explicit; the $N = 3$ base case, where the chain is empty, is checked by the kernel.) □

5 The sharp upper bound: $f(N) \le N + 4$

This is the mathematical core. The formal statement (`state_law_N_add_four`) is exactly the upper-bound half of Theorem 1.1; its proof occupies most of the development. What follows is a faithful map of that argument: every definition and numbered claim below corresponds to a formal one, and the file named in brackets is where it lives. Proofs here are proof *sketches* — the intended level is "the reader sees why it is true and what must be checked"; the formal text is the complete verification.

Throughout, fix a wiring w on switches $0, \dots, N-1$.

5.1 Trailing cascades and the retrace lemma

The engine of everything is the asymmetry of the lazy point: trailing passes write, facing passes read.

Lemma 5.1 (Cascades are wiring-determined; GeneralN.lean)**.** *A trailing entry exits by the stem regardless of the tongues. Hence a maximal run of consecutive trailing passes (a* cascade*) — its route, and the pins it writes — depends only on the wiring and the entry port, not on the tongue vector.*

Lemma 5.2 (Retrace; GeneralN.lean)**.** *After a cascade, enter the* stem *of its last switch with any tongue vector agreeing with the cascade's pins. Then the walk performs pure facing moves that traverse the cascade backwards, touches no tongue, and emerges over the cascade's original entry edge.*

Sketch. Each cascade pass pinned its switch's tongue toward the branch the train entered by. Arriving later at that switch's stem, the tongue therefore selects exactly that branch: the train is routed back along the edge it came in by, facing, changing nothing. Induction along the cascade. □

This one-switch fact — a passage leaves the points set so that immediately traversing it backwards is a no-op (`arrive_back`, TrackTrace.lean) — is why grooves, teardrops bounce, and long paths are re-walked for free. We say a path is *grooved* at a vector u when every passage of the path is the one u selects, and call a path *switch-simple* if it visits each switch at most once. A switch-simple path has at most N passages, and any tongue changes it makes survive to its endpoint.

5.2 First revisit: cycles and manufactured reflectors

Follow the train from its start and record the *physical trace*: the sequence of local passages. Because there are finitely many switches, some switch is eventually revisited; the analysis of the *first* revisit is the fork in the whole proof (TrackTrace.lean, TrackLobe.lean, TripleSelfLinkSimple-CycleClosure.lean).

Lemma 5.3 (First-revisit dichotomy)**.** *Consider the first time the switch-simple initial exploration re-enters a switch it has already visited. Up to the symmetric cases, one of:*

1. *the walk closes a* simple cycle *and settles on it: after a switch-simple transient, the trajectory becomes periodic over a switch-simple cycle whose vector no longer changes; or*

2. *the walk closes a* lobe *(re-enters the old switch by its other branch) or bounces off a* self-link/cap*: the layout has* manufactured a reflector*.*

Definition 5.4 (Manufactured reflectors; TrackTheta.lean)**.** A *manufactured reflector* on entry edge $e \to g$ consists of a switch-simple *runway* from g to a *mouth*, together with either

- (*stay reflector*) a self-linked arm at the mouth — the cap reflects the train back through the branch it came by, re-pinning the same tongue: the local action is the identity; or
- (*flip reflector*) a lobe at the mouth (the *candy*) — the far contact returns the train through the other branch, so each complete traversal flips exactly the mouth switch's tongue (the *action switch*).

In both cases the return journey is the runway retraced backwards (Lemma 5.2): a complete traversal leaves every tongue unchanged except possibly the one action switch, and exits over the entry edge e. The switches of runway and candy are the reflector's *reusable support*; the action switch of a flip reflector is deliberately *not* counted in it.

The state-side consequences are the first two entries of the budget:

Lemma 5.5 (Construction history; ManufacturedPairNovelty.lean)**.** *The complete first manufacturing journey — switch-simple exploration, contact, and reverse runway — exhibits at most $N+2$ distinct restricted vectors: the at most $N+1$ vectors along the switch-simple exploration (one new tongue write per step at distinct switches, starting from the initial vector), plus at most one for the activated state. The activation itself, however long the runway, contributes at most that single new vector, because the return is a retrace.*

Lemma 5.6 (Two-phase and four-corner laws; ManufacturedPairNovelty.lean, PairActionCorners.lean)**.** *A complete traversal of a manufactured reflector has at most two tongue phases: the incoming vector, and that vector with the local action applied. For* two *opposite reflectors A, B bouncing a train between them, every vector at every time lies in the four algebraic corners*

$$u, \quad u+A, \quad u+B, \quad u+A+B,$$

whether or not their supports intersect. At either boundary, both supports are grooved at a reference corner and the current vector is that corner or the previous action applied to it. A normal traversal, stem capture, or branch repair preserves this invariant. Strong induction over positive-length excursions gives the all-time cover; no runway length enters the number of corners.

The dogbone of Section 3 is the equality case of Lemma 5.6, and the cap-versus-lobe distinction is exactly the sticky/alternating dichotomy visible in the toy.

5.3 The coordinate budget

Call a live step *productive* when it changes the restricted vector; its *writer* is the switch it enters (TrackFiniteAlternation.lean). Record the initial vector and the post-vector of every productive step. This history covers every live prefix by induction: a quiet step repeats its preceding vector, and a productive step adds its own post-vector (SharpCertificateClosure.lean).

On a switch-simple trace the writer labels are injective, so the productive steps already have distinct writers. No separate first-writer predicate is needed. If the continuation's endpoints groove the old reusable support, its productive writers avoid that support. Thus one joint coordinate budget pays for the old support, the continuation writers, and any disjoint reserved coordinates:

$$\#\{\text{old reusable switches}\} + \#\{\text{productive writers}\} + \#\{\text{reserved switches}\} \le N.$$

The first history omits a known duplicate, and the continuation erases their shared boundary vector. The same history-size proof handles either an empty reserve or the old action switch, leaving only the explicitly named constant tail corners (TwoHistoryUnionCharge.lean, OneReflectorContinuation.lean).

The endpoint-coordinate law also shortens this bookkeeping. On a switch-simple trace, each intermediate coordinate equals its initial or final value. Thus a productive writer cannot agree at the two endpoints: its values immediately before and after the write would both equal that common value. If the endpoints differ only at one coordinate, every intermediate vector is one of the endpoints, since all other coordinates are fixed and the remaining coordinate selects an endpoint (TrackTrace.lean, TrackLobe.lean).

5.4 The probe tree after one reflector

Assume now the wiring is total on the first N switches and the run's *incoming edge is known*. Section 5.5 constructs such a comparison wiring for every original partial wiring, preserving all originally live configurations. Totality makes both probes live. The first-revisit dichotomy (Lemma 5.3) and a second probe of $N+1$ raw steps after the first reflector generate a finite tree of outcomes, each closed by its own count (files PartialSecondRunSharp.lean, OneReflectorContinuation.lean, TripleSelfLinkSimpleCycleClosure.lean):

outcome	count	mechanism
settles on a simple cycle	$\le N+2$	transient + repeat and settled vectors
reflector, then stable cycle	$\le N+3$	joint charge of support and writers
reflector, then changed contact	$\le N+4$	Lemma 5.7
two opposite protected reflectors	$\le N+4$	Lemma 5.8

The two $N+4$ leaves are closed directly in StateLawNAddFourSharp.lean; everything else in the tree is $N+3$ or less.

Lemma 5.7 (Changed contact; ChangedContactNAddFour.lean)**.** *Suppose after the first flip reflector the run makes a first support-changing contact. The general changed-support count pays a compressed lead of $N+3$ plus two fresh Gray corners; but the reflector's facing action switch is deliberately absent from its reusable support, so unless the strict pre-contact approach productively first-writes that very switch, the action switch is a reserved ambient coordinate, the lead compresses to $N+2$, and the branch closes at $N+4$. In the remaining action-written case the local forward tail has only one fresh corner: the constant runway retrace makes that action write the last productive write and recovers its historical pre-write vector. Again the count is $N+3+1=N+4$.*

Lemma 5.8 (Protected pair; ProtectedPairNAddFour.lean, PairActionCorners.lean)**.** *Suppose the second probe manufactures an opposite reflector, with the old support grooved at both endpoints of the second construction. Use the same canonical construction history in every case, and view* *the continuation as a suffix of the grooved pair run started at the protected pre-return state: the four-corner theorem confines every later sample to the pre-return and activated states and their two images under the old reflector's action, of which the first two are historical. If the old reflector is a stay reflector, all four corners are historical, so the generic $N+3$ history bound suffices. For a flip reflector, split only on whether its private action coordinate occurs among the second construction's productive first writers. If absent, reserve that coordinate: again the budget is $(N+2)+2$. If present, its write forces a constant-tongue retrace and is the last productive writer. Its recorded pre-write state accounts for a nominally fresh tail corner, giving $(N+3)+1$. In every case the total is at most $N+4$; the earlier first-writer-order subdivision and doubly-erased history are unnecessary.*

The continuation history and its optional reserved-coordinate list supply both budgets. A pointwise cover passes through each manufacturing journey in turn. The keystone `known_edge_N_add_four` closes both probe outcomes directly; `state_law_N_add_four` then transfers this bound to arbitrary partial wirings.

Theorem 5.9 (Known incoming edge; StateLawNAddFourSharp.lean)**.** *On a wiring total below $3N$, if the start is entered over a wired edge, every duplicate-free list of live sample times with pairwise-distinct restricted vectors has length at most $N+4$.*

5.5 Arbitrary starts, by completing every free port

A general start need not arrive over an edge, and a partial wiring may let its run fall off. Both complications can be removed in one comparison. The abstract model admits self-links, so completing an unwired port costs no switch. No totality premise is added to the headline theorem.

Lemma 5.10 (Total completion; WiringCompletion.lean)**.** *For any wiring w using only switches $0, \ldots, N-1$, define v by preserving every existing edge and setting $v(p) = p$ at every unwired port $p < 3N$. Then v is a symmetric wiring using the same switches, every port below $3N$ has a partner, and every in-range start has a live configuration after any finite number of steps.*

Proof. No old edge can point to a free port: symmetry of w would make that port wired already. The new self-links therefore preserve symmetry and do not modify any old edge. Their endpoints are below $3N$. A local passage exits on the same switch as it entered, hence remains in range; its exit now always has a partner, also in range. Induction gives liveness at every finite time. □

Lemma 5.11 (Extension preserves live runs; `stepN_preserved_by_wiring_extension`, WiringCompletion.lean)**.** *If every edge of w is an edge of v, every configuration reached after n live steps in w is reached at exactly time n in v.*

Proof. Induct on n. A successful original step follows an unchanged edge, with the same local tongue update. The statement makes no claim about the continuation after the original run dies, which is where the completion may begin to differ. □

Theorem 5.12 (Sharp upper bound; `state_law_N_add_four`, StateLawNAddFourSharp.lean)**.** *For every wiring on switches $0, \ldots, N-1$, every start, and every duplicate-free list of live sample times with pairwise-distinct restricted vectors, the list has length at most $N+4$.*

Proof. If the starting port p is at least $3N$, its first exit is on the same out-of-range switch and cannot be wired. Only time zero can be live, contributing at most one vector. Otherwise complete every free port by Lemma 5.10. The completed wiring is total, symmetry supplies an incoming edge at the original starting port, and Theorem 5.9 applies. By Lemma 5.11, every original live sample carries the identical vector at the identical time in the completion. Transfer its $N + 4$ bound back to the original sample list, including time zero. □

5.6 Selected routes and boundary invariants

A reflector's selected outward route already encodes the direction in which its loop is traversed. The first occurrence of a disturbed support coordinate suffices for the contact argument; the general route theorem allows later repetitions of any switch. Before that first contact the disturbed coordinate is untouched. At a branch entry the lazy-point rule restores the reference value; after that passage both runs have the same full configuration, so determinism identifies every subsequent state. At a stem entry the mouth-capture law applies. There is no separate runway, forward-loop, or reverse-loop contact classification (RunwaySpliceNovelty.lean, PairActionCorners.lean).

For all-time covers it is enough to close a boundary invariant under positive-length excursions whose intermediate tongue vectors belong to the claimed cover. Strong induction on the queried time proves the claim: a query inside the first excursion uses its local cover; a later query subtracts the strictly positive duration and continues at the next invariant boundary. One invariant now handles every manufactured pair. Let $S = \{s, a(s), b(s), b(a(s))\}$ be the four-corner orbit of the two commuting involutions. A boundary configuration has a reference $u \in S$ that grooves both supports, with its current vector in

$$\{u, b(u)\} \quad \text{at } g, \qquad \{u, a(u)\} \quad \text{at } e.$$

If the next reflector remains grooved, its ordinary traversal uses the current vector as the new reference. Otherwise the previous action must be a flip, and the first support contact either captures back to u or repairs the fault and completes the next traversal from u. All intermediate vectors stay in S, and each resulting boundary has the same form. This replaces the separate stay/flip, one-sided, and mutual-contact arguments (PairActionCorners.lean).

An arbitrary mouth-free lobe has a grooved route in either orientation. Pin the current mouth tongue to its recorded entry branch: the current vector is that pinned vector or its mouth flip. All interior grooves remain unchanged. A stay splice repeats a simpler two-phase excursion: traverse the old stay suffix without changing the vector, then traverse the lobe and flip its mouth. At a self-linked boundary the unchanged prefix has length zero. The lobe makes either excursion positive, and its mouth flip preserves the same two-vector family (PartialSecondRunSharp.lean).

For a flip reflector opposite an arbitrary lobe, one invariant now handles every excursion. At the old reflector's outer boundary, both supports are grooved and the vector lies in their four-corner orbit. Select the lobe's route in the current vector; this route represents every interior switch even when its orientation is reversed. If the old action avoids the route, both components traverse normally. At a contact, the shared first-contact theorem either captures and restores the reference vector or synchronizes with its remaining traversal. Both outcomes restore the same boundary invariant after a positive excursion. Strong induction therefore gives the all-time cover without constructing a separate reverse excursion. The corner-closure calculation is shared with the manufactured-pair argument (TrackNormalForm.lean, RunwaySpliceNovelty.lean).

The protected facing exits need no separate approach-and-return invariant. The continuation after the protected pre-return state is a suffix of the grooved pair run started there, so the four-corner theorem of Lemma 5.6 covers it pointwise: only the pre-return and activated states and their two images under the old action can appear (PairActionCorners.lean).

Backward contacts and same-exit cycles use first-arrival synchronization. If $\mathrm{arrive}(u, p) = (x, v)$, the local switch law gives $\mathrm{arrive}(v, p) = (x, v)$ too: both starts have the same complete successor and therefore the same configuration at every positive time. A nonempty closed spatial route grooved by v replays forever without changing v, even if its interior repeats switches. When the contact supplies such a route, the original run consequently has vector u at time zero and v at every positive time. There is no need to analyse a transient lap and then reduce time modulo a stable period (TrackNovelReplay.lean, TripleSelfLinkSimpleCycleClosure.lean).

5.7 Closed spatial routes and one variable tongue

For a strict splice inside the old reflector's loop, the new splice tongue is latched. The remaining spatial lap consists of the old loop completion, the reverse runway, the fresh approach, and the final splice contact. Only the old reflector's action tongue can vary. The facing-approach contradiction already proved in TrackGlobalRepair.lean shows that the lap never enters that variable switch through its stem. A branch entry may set the variable tongue, but its exit is always the stem. Every other passage is grooved in both possible vectors.

Therefore both values of the variable tongue follow the same spatial lap, and each passage preserves the same two-vector family. The recorded starting and finishing tongues of the lap need not agree. Induction replays any recorded route in a passage-local invariant; the positive-length progress principle then repeats a closed spatial route indefinitely. This gives the all-time two-vector bound directly (TrackTrace.lean, ForeignSpliceNovelty.lean). There is no need to distinguish approaches that avoid the old action from approaches that repair it, or to construct their separate transient and periodic windows.

5.8 Reuse recorded witnesses and trace suffixes

The construction state stored in a manufactured reflector is not the later state in which its support is grooved. Its traversal law already works in any such later state. Trimming a runway can therefore retain the original loop trace, mouth, terminal states, and crossing proof, replacing only the runway prefix and its initial state and incoming edge. Switch simplicity passes to the suffix, and its smaller support remains grooved in the later state. Rebuilding the witness in that later state, including a new forward/reverse loop split, is unnecessary (TrackGlobalRepair.lean).

More generally, grooved trace replay is state-independent. A contact exiting through a recorded trace's endpoint retraces it and leaves over its incoming edge: induction reverses the tail and then the head. This one fact covers reverse loops, reverse arbitrary lobes, and final runway returns, including empty paths.

6 Assembly

Proof of Theorem 1.1. Upper bound: there are only 2^N Boolean tongue vectors (`state_law_two_pow`), and a duplicate-free sample list is also bounded by $N + 4$ (Theorem 5.12), hence by their minimum. *Attainment:* for $N = 0$ the empty layout, for $N = 1$ the teardrop, for $N = 2$ the dogbone (Section 3), each attaining $2^N = \min(2^N, N + 4)$; for $N \geq 3$ the extremal family (Theorem 4.2) attains $N + 4$, which is the minimum since $N + 4 \leq 2^N$ for $N \geq 3$ (`add_four_le_two_pow`). □

7 Beyond the count: a conjectured structure theorem

The state law counts vectors. Experiments with an independent simulator of the model (a direct port of Definition 2.2, used to check every table in this paper) suggest that the count is the shadow of a sharper *shape*. Call the vectors of a run, in order of first appearance, $v_1, v_2, \ldots$.

> **Conjecture (square steady state).** Let d be the number of switches that ever change after the $(N{+}1)$-th distinct vector has appeared. Then $d \leq 2$: from v_{N+1} on, every vector of the run lies in the d-dimensional subcube spanned by those d switches — for $d = 2$, the Gray square $\{v,\ v \oplus a,\ v \oplus b,\ v \oplus a \oplus b\}$. Consequently the run has at most $N + 2^d \leq N + 4$ distinct vectors.

Evidence. Over *all* symmetric partial pairings of the $3N$ ports (self-links included), all start ports, and all initial tongue vectors — 2.1 million runs at $N = 3$ — the conjecture holds with every bound attained: the maximum count is $7 = N + 4$, at most 3 repeated-writer novelties occur, the tail dimension never exceeds 2, and in the worst case the square begins exactly at v_{N+1}, so "$N + 1$" cannot be improved to "N". Random layouts at $N = 4, 5, 6$ (six hundred thousand runs) agree in every respect, with maxima $8, 9, 10$. The dependence $N + 2^d$ is exact in the data: tails of dimension $0, 1, 2$ reach $N + 1$, $N + 2$, $N + 4$ vectors respectively and no more.

Relation to the formal proof. In the protected-pair regime the square is already a theorem: `manufactured_pair_all_time_action_corners_tongues` (PairActionCorners.lean) proves that *at every time* the phase of any manufactured reflector pair is one of the four corners $u, u{+}A, u{+}B, u{+}A{+}B$, whatever the supports do. The two moving switches are the mouths of the two reflectors the run manufactures — the dogbone of Section 3 in its universal form — and the remaining leaves of the probe tree (Section 5.4) bound their tails by budgets that are, in every case, flips of at most two named switches. A proof of the conjecture along the existing lines would therefore restate each leaf's count as a cover by a square; a proof from first principles would have to show directly that once the switch-simple exploration is spent, the walk can only ever re-write two switches. Either would explain *why* the constant is four: it is the size of a square.

8 About the formalisation

The development in theory/lean builds with Lean 4.32.2 and no external library. Run `lake build` to compile all 31 Lean files; StateLawAxiomAudit.lean checks the exact transitive axiom list of `GeneralN.state_law`:

```
[propext, Classical.choice, Quot.sound].
```

There is one headline theorem, in StateLaw.lean. Every explicitly declared public support theorem is used in its kernel dependency closure. There is no `sorry` or `native_decide`: the finite witness runs of Section 3 and the $N = 3$ base of Theorem 4.2 are checked by kernel `decide`. No definition uses choice to produce data; there is no `noncomputable` declaration. Classical reasoning occurs only in proofs. The upper bound and the $N \geq 4$ attainment proof are symbolic in N, with structural induction rather than finite-instance enumeration.

The extremal family was found empirically. Physical track geometry never enters the formal dynamics: ordinary track merely transports the train between switch ports. The main source entry points are:

entry point	contents
StateLaw.lean	the theorem and its statement language
StateLawSmallN.lean	2^N ceiling; empty/teardrop/dogbone
StateLawLowerBound.lean	the extremal family, $N \geq 3$
StateLawNAddFourSharp.lean	the total known-edge theorem and the sharp $N+4$ bound
WiringCompletion.lean	total completion and preservation of live runs
TrackTrace.lean, TrackTheta.lean	traces, reflectors, theta engine
StateLawAxiomAudit.lean	the axiom audit